

Azure Service Bus – Receiving a Message from a Queue Using C

1. What We Will Learn

In this example, we will learn how to **receive a message from an Azure Service Bus Queue using C#**.

We will use:

1. `ServiceBusClient`
2. `ServiceBusReceiver`
3. `ReceiveMessageAsync()`

The overall flow is:

```
Azure Service Bus Queue
    |
    | Message
    v
ServiceBusReceiver
    |
    v
C# Console Application
```

2. Prerequisites

Before starting, we need:

- Azure Service Bus Namespace
 - Azure Service Bus Queue
 - Shared Access Policy
 - Connection String
 - Queue Name
 - Visual Studio
 - .NET Console Application
-

3. Install Azure Service Bus NuGet Package

Create a new **Console Application** in Visual Studio.

Example project name:

```
ServiceBusReceiver
```

Install the NuGet package:

```
Azure.Messaging.ServiceBus
```

Steps

```
Visual Studio
  ↓
Project
  ↓
Manage NuGet Packages
  ↓
Browse
  ↓
Azure.Messaging.ServiceBus
  ↓
Install
```

This package provides the classes required to communicate with Azure Service Bus.

Important classes:

```
ServiceBusClient
ServiceBusSender
ServiceBusReceiver
ServiceBusProcessor
ServiceBusMessage
```

4. What Do We Need to Receive a Message?

To receive a message from a queue, we need:

1. Connection String
2. Queue Name

Then the basic process is:

```
Connection String
    ↓
ServiceBusClient
    ↓
ServiceBusReceiver
    ↓
Receive Message
```

5. Step 1 – Get the Connection String

Go to the Azure Portal.

Navigate to:

```
Azure Portal
    ↓
Service Bus Namespace
    ↓
Queue
    ↓
Settings
    ↓
Shared access policies
    ↓
Open the required policy
    ↓
Copy Primary Connection String
```

The connection string allows the C# application to connect/authenticate with Azure Service Bus.

Example:

```
string connectionString = "<SERVICE_BUS_CONNECTION_STRING>";
```

6. Step 2 – Get the Queue Name

We also need the name of the queue from which we want to receive the message.

Example:

```
string queueName = "<QUEUE_NAME>";
```

The queue name tells the receiver which queue it should listen to.

7. Step 3 – Create ServiceBusClient

First, create the `ServiceBusClient`.

```
ServiceBusClient serviceBusClient =  
    new ServiceBusClient(connectionString);
```

What is ServiceBusClient?

`ServiceBusClient` is used to establish a connection between our C# application and Azure Service Bus.

```
C# Application  
    |  
    v  
ServiceBusClient  
    |  
    v  
Azure Service Bus
```

Remember:

ServiceBusClient = Connection

8. Step 4 – Create ServiceBusReceiver

Now create a receiver using the `ServiceBusClient`.

```
ServiceBusReceiver serviceBusReceiver =  
    serviceBusClient.CreateReceiver(queueName);
```

Here:

- `serviceBusClient` → connects to Service Bus
- `CreateReceiver()` → creates a receiver
- `queueName` → specifies the queue from which we want to receive messages

Flow:

```
ServiceBusClient  
    |  
    | CreateReceiver(queueName)  
    v  
ServiceBusReceiver  
    |  
    v  
Queue
```

9. Step 5 – Receive the Message

Now use `ReceiveMessageAsync()` to receive a message.

```
ServiceBusReceivedMessage message =  
    await serviceBusReceiver.ReceiveMessageAsync();
```

The flow is:

```
Azure Service Bus Queue  
    |  
    | Message  
    v  
ServiceBusReceiver  
    |  
    | ReceiveMessageAsync()  
    v  
ServiceBusReceivedMessage
```

10. Step 6 – Read the Message Body

After receiving the message, we can access its body.

For example:

```
Console.WriteLine(message.Body.ToString());
```

If the queue contains:

```
Hello from C# and Visual Studio
```

the output will be:

```
Hello from C# and Visual Studio
```

11. Complete Basic Code

The complete example looks like this:

```
using Azure.Messaging.ServiceBus;

string connectionString = "<SERVICE_BUS_CONNECTION_STRING>";
string queueName = "<QUEUE_NAME>";

ServiceBusClient serviceBusClient =
    new ServiceBusClient(connectionString);

ServiceBusReceiver serviceBusReceiver =
    serviceBusClient.CreateReceiver(queueName);

ServiceBusReceivedMessage message =
    await serviceBusReceiver.ReceiveMessageAsync();

Console.WriteLine("Message has been received");

Console.WriteLine(message.Body.ToString());
```

12. Understanding the Code Step-by-Step

Step 1 – Import the Package

```
using Azure.Messaging.ServiceBus;
```

This gives access to Azure Service Bus classes.

Step 2 – Connection String

```
string connectionString = "<SERVICE_BUS_CONNECTION_STRING>";
```

This contains the information required to connect/authenticate with Azure Service Bus.

Step 3 – Queue Name

```
string queueName = "<QUEUE_NAME>";
```

This identifies the queue from which we want to receive the message.

Step 4 – Create Client

```
ServiceBusClient serviceBusClient =  
    new ServiceBusClient(connectionString);
```

Creates the Service Bus client.

Remember:

Client = Connect

Step 5 – Create Receiver

```
ServiceBusReceiver serviceBusReceiver =  
    serviceBusClient.CreateReceiver(queueName);
```

Creates a receiver for the specified queue.

Remember:

Receiver = Receive

Step 6 – Receive Message

```
ServiceBusReceivedMessage message =  
    await serviceBusReceiver.ReceiveMessageAsync();
```

Receives a message from the queue.

Step 7 – Read Message Body

```
message.Body.ToString()
```

Gets the body/content of the received message.

13. What is ServiceBusReceivedMessage?

When we receive a message, Azure Service Bus returns a:

```
ServiceBusReceivedMessage
```

This object contains the message and its metadata.

For example:

```
ServiceBusReceivedMessage  
|
```



```
+----- Body
|
+----- MessageId
|
+----- Subject
|
+----- CorrelationId
|
+----- ContentType
|
+----- Application Properties
```

The most important property for beginners is:

```
message.Body
```

because this contains the actual message data.

14. Example

Suppose the sender sent:

```
ServiceBusMessage message =
    new ServiceBusMessage("Hello from C# and Visual Studio");
```

The message is stored in the queue.

```
Sender
|
| "Hello from C# and Visual Studio"
v
Azure Service Bus Queue
```

The receiver then reads it:

```
ServiceBusReceivedMessage message =
    await serviceBusReceiver.ReceiveMessageAsync();
```

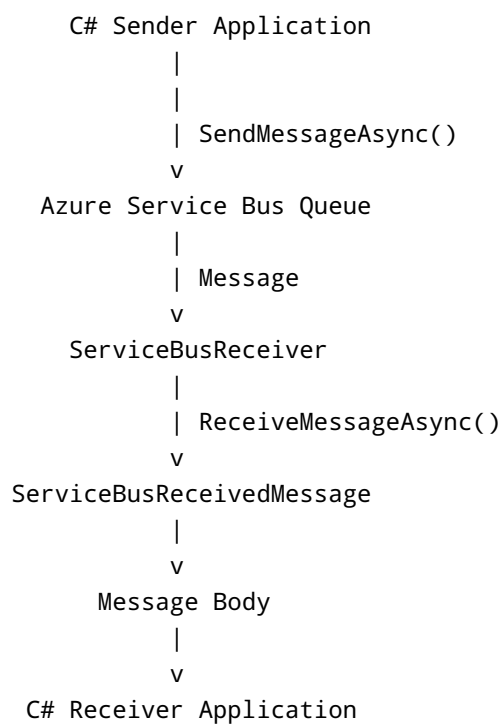
And gets the body:

```
message.Body.ToString()
```

Result:

```
Hello from C# and Visual Studio
```

15. Complete Message Flow



16. How to Verify the Message in Azure Portal

Before running the receiver, we can verify that a message exists in the queue.

Go to:

```
Azure Portal  
↓  
Service Bus Namespace
```

↓
Queue
↓
Service Bus Explorer
↓
Peek messages

Select:

Pick from start

Then we can see the message:

Hello from C# and Visual Studio

This confirms that there is a message waiting in the queue.

17. What Happens When We Receive the Message?

This is an important concept.

When we call:

```
await serviceBusReceiver.ReceiveMessageAsync();
```

the receiver gets a message from the queue.

However, **receiving a message and successfully completing a message are separate concepts.**

Azure Service Bus supports different message settlement operations.

The important ones are:

Complete
Abandon
Dead-letter
Defer

For a basic example, remember:

Receiving a message does not automatically mean that the business processing has successfully completed.

18. Peek vs Receive

This is very important when testing through Service Bus Explorer.

Peek

Peek allows us to inspect a message without removing it from the queue.

```
Queue
|
| Peek
v
View Message
```

The message remains available.

Receive

Receive retrieves the message for processing.

```
Queue
|
| Receive
v
Application
```

Depending on the receive/settlement mode and subsequent settlement operation, the message can be completed or become available again.

Easy way to remember

```
Peek    = Look at message
Receive = Get message for processing
```

19. Why Do We Use `await`?

We use:

```
await serviceBusReceiver.ReceiveMessageAsync();
```

because `ReceiveMessageAsync()` is an asynchronous operation.

`await` means:

Wait for the asynchronous receive operation to complete without blocking the current thread.

Example:

```
ReceiveMessageAsync()  
    |  
    | Waiting for message  
    v  
Message received  
    |  
    v  
Continue execution
```

20. Sender vs Receiver

This is an important interview question.

Sender

Used to send messages.

```
C# Application  
    |  
    v  
ServiceBusSender  
    |  
    v  
Queue
```

Receiver

Used to receive messages.

```
Queue
|
v
ServiceBusReceiver
|
v
C# Application
```

Easy Memory Trick

```
Sender    → Send
Receiver  → Receive
```

21. ServiceBusReceiver vs ServiceBusProcessor

Both can be used for receiving messages, but their programming models are different.

ServiceBusReceiver

You explicitly call:

```
await receiver.ReceiveMessageAsync();
```

Think:

"I want to receive a message now."

ServiceBusProcessor

You register a message handler:

```
Message arrives
↓
Processor
```

↓
Handler executes

Think:

"Keep listening and process messages whenever they arrive."

22. Real-Time Example

Suppose we have an Order Queue.

```
Order Service
  |
  | Send Order
  v
Order Queue
  |
  | Receive
  v
Payment Service
```

The Payment Service can create a receiver:

```
ServiceBusReceiver receiver =
    client.CreateReceiver("OrderQueue");
```

Then receive the order:

```
ServiceBusReceivedMessage message =
    await receiver.ReceiveMessageAsync();
```

Then read the order data:

```
string orderData =
    message.Body.ToString();
```

Then process the payment.

23. Interview Questions

Q1. How do you receive a message from Azure Service Bus using C#?

Answer

First, create a `ServiceBusClient` using the connection string. Then create a `ServiceBusReceiver` for the required queue using `CreateReceiver()`. Finally, call `ReceiveMessageAsync()` to receive the message and access its body using the `Body` property.

Flow:

```
Connection String
    ↓
ServiceBusClient
    ↓
ServiceBusReceiver
    ↓
ReceiveMessageAsync()
    ↓
ServiceBusReceivedMessage
    ↓
Message Body
```

Q2. What is `ServiceBusReceiver`?

`ServiceBusReceiver` is used to receive messages from an Azure Service Bus queue or topic subscription.

Q3. What method is used to receive a message?

```
ReceiveMessageAsync()
```

Example:


```
ServiceBusReceivedMessage message =  
    await receiver.ReceiveMessageAsync();
```

Q4. How do you get the message body?

```
message.Body.ToString()
```

Q5. What is `ServiceBusReceivedMessage`?

`ServiceBusReceivedMessage` represents a message received from Azure Service Bus and contains both the message body and message metadata.

Q6. What is the difference between Peek and Receive?

Peek allows us to inspect a message without removing it for processing. **Receive** retrieves the message for processing according to the configured receive and settlement behavior.

24. Important Concepts to Remember

ServiceBusClient

```
Connect
```

ServiceBusReceiver

```
Receive
```

ReceiveMessageAsync()

```
Receive message asynchronously
```

ServiceBusReceivedMessage

Received message + metadata

Body

Actual message content

Peek

View without consuming

25. Quick Revision

Remember this sequence:

```
1. Connection String
   ↓
2. ServiceBusClient
   ↓
3. CreateReceiver(queueName)
   ↓
4. ReceiveMessageAsync()
   ↓
5. ServiceBusReceivedMessage
   ↓
6. message.Body
```

Memory Trick

Client → Receiver → Receive → Read

```
CLIENT   = Connect
RECEIVER  = Receive
MESSAGE   = Received message
BODY      = Actual data
```

One-Line Interview Answer

To receive a message from an Azure Service Bus Queue in C#, we create a `ServiceBusClient` using the connection string, create a `ServiceBusReceiver` using the queue name, call `ReceiveMessageAsync()`, and then read the message content from `ServiceBusReceivedMessage.Body`.